



Learning objectives

By the end of this project, you will be able to:

- Load and profile real-world data read a messy dataset with the correct encoding, then use head, shape, info, describe, and value_counts to assess its size, structure, and quality.
- Apply the full pandas wrangling toolkit demonstrate all 21 data-cleaning techniques (creating, selecting, filtering, sorting, replacing, renaming, handling missing values, dropping rows/columns/duplicates, grouping, aggregating, applying functions, resampling, concatenating, and merging) on a single dataset.
- Diagnose and resolve common data problems identify and handle missing values, duplicate rows, inconsistent categories, invalid prices/quantities, and cancellations/returns.
- Make and justify cleaning decisions choose defensible strategies (what to drop, keep, or repair), keep raw and cleaned data separate, and treat dataframes as immutable objects.
- Turn clean data into insight engineer features and aggregate the data to answer real business questions about sales, products, markets, and customers.
- Communicate to a non-technical audience present finding clearly with short interpretations and at least two supporting charts.
- Produce reproducible, version-controlled work document your process and manage your code and outputs in a github repository that another person can clone and run end to end.

The scenario

You have just joined the analytics team at **Cobblestone Gifts**, a UK-based online retailer that sells all-occasion novelty gifts to both wholesale buyers and individual shoppers. The company has never run a formal sales review. With the financial year closing, the Head of Commercial wants a clear picture of what sold, to whom, and where but the only thing the IT team can hand you is a single raw export pulled straight from the order system: roughly half a million transaction lines from December 2010 to December 2011.

The export has never been touched. It mixes completed sales with **cancelled orders and returns**, is missing a customer ID on a large share of rows, contains **blank product descriptions** and **duplicate lines**, and is peppered with non-product entries such as postage, bank charges, and manual adjustments. Prices and quantities include impossible values. In short: **no number in this file can be trusted yet.**

Your job is to clean it. You will profile the data, make and document defensible cleaning decisions, build a tidy *completed-sales* dataset, and then use it to answer a set of real business questions. The Head of Commercial will read your findings so every claim must rest on data you can stand behind.

The dataset

E-Commerce Data (UK Online Retail), a real, anonymised dataset of transactions from a UK online gift retailer (originally compiled at London South Bank University and hosted on the UCI ML Repository).

Kaggle page: <https://www.kaggle.com/datasets/carriell/ecommerce-data>

How to download

- **Option A — website:** open the Kaggle page above, click **Download**, and unzip. You will get a single file named `data.csv` (~45 MB, 541,909 rows).
- **Option B — Kaggle API:** install with `pip install kaggle`, place your `kaggle.json` token in `~/.kaggle/`, then run `kaggle datasets download -d carriell/ecommerce-data` and unzip.

Gotcha you must handle: the file is not UTF-8. Load it with `pd.read_csv('data.csv', encoding='ISO-8859-1')` or it will raise a decoding error on the £ symbol.

Column dictionary

Column	Type	Meaning & cleaning relevance
InvoiceNo	text	Transaction ID. A leading 'C' marks a cancellation — these must be separated from sales.
StockCode	text	Product code. Watch for non-products: POST, DOT, M, BANK CHARGES, AMAZONFEE.
Description	text	Product name. ~1,454 are blank ; spacing/case is inconsistent.
Quantity	integer	Units per line. Negative values indicate returns/cancellations.
InvoiceDate	text	Timestamp stored as text — must be parsed to datetime for any time analysis.
UnitPrice	float	Price per unit (£). Includes zero and negative values that aren't valid sales.
CustomerID	float	Customer number. ~135,080 rows are missing it (~25%) — a key decision point.
Country	text	Customer country. Inconsistent labels (e.g. <i>EIRE</i> , <i>RSA</i> , <i>Unspecified</i>).

Part 1 Clean the data

Work through the phases below. **Every technique is required and must be clearly labelled in your notebook** (e.g. a markdown header `### 3.13 Dropping duplicates`) so it is easy to grade. Keep your raw DataFrame untouched and build your cleaned version as a new object.

PHASE A · Load & profile

#	Technique	What to do with the retail data
3.2	Getting information	Load data.csv with the right encoding. Report <code>shape</code> , and run <code>head()</code> , <code>info()</code> , <code>describe()</code> . State which columns have missing values and how many.
3.9	Unique values	Run <code>value_counts()</code> / <code>nunique()</code> on Country, StockCode, Description. Surface non-product codes and use <code>value_counts(dropna=False)</code> to expose missing entries.
3.8	Min/max/sum/mean/count	Summarise Quantity and UnitPrice. Identify the negative quantities (returns) and the zero/negative prices that cannot be valid sales.
3.1	Creating a DataFrame	Build a small country → region lookup table from a Python dictionary (e.g. United Kingdom → UK&IE; France, Germany → Western Europe). You will merge it in later.

PHASE B · Select & filter

#	Technique	What to do with the retail data
3.3	Slicing	Inspect row ranges with <code>iloc</code> . Create a unique line-item index and retrieve rows by label with <code>loc</code> .
3.4	Conditional selection	Isolate cancellations (InvoiceNo starts with 'C'), rows where Quantity <= 0, and rows where UnitPrice <= 0. Count each group.
3.5	Sorting values	Sort by Quantity and by line revenue to find the largest bulk orders and the largest returns.

PHASE C · Clean & fix

#	Technique	What to do with the retail data
3.6	Replacing values	Standardise country labels: 'EIRE' → 'Ireland', 'RSA' → 'South Africa', 'Unspecified' → missing.
3.7	Renaming columns	Rename all columns to snake_case (InvoiceNo → invoice_no, UnitPrice → unit_price, ...).
3.10	Handling missing values	Quantify missingness, then decide and justify a policy for missing CustomerID and blank Description. Demonstrate <code>isnull()</code> , <code>dropna()</code> , <code>fillna()</code> .
3.11	Deleting a column	Drop a column you will not use in the final analysis and explain why.
3.12	Deleting a row	Remove cancellations, non-product lines, and impossible prices/quantities to build a clean sales DataFrame.

3.13	Dropping duplicates	Detect exact duplicates with <code>duplicated().sum()</code> and remove them with <code>drop_duplicates()</code> . Report how many rows were dropped.
------	---------------------	---

PHASE D · Engineer & summarise

#	Technique	What to do with the retail data
3.18	Applying a function	Add a <code>revenue = quantity * unit_price</code> column. Use <code>apply</code> to clean Description (strip + title-case) and to flag cancelled invoices.
3.17	Looping over a column	Show a for-loop / list comprehension over a column, then explain why <code>apply</code> is the preferred, faster approach.
3.14	Grouping by values	Use <code>groupby</code> to compute revenue by Country and number of orders per customer.
3.16	Aggregating	Use <code>agg</code> to produce several statistics at once (total revenue, mean order value, transaction count) per country and per customer.
3.19	Applying to groups	Use <code>groupby(...).apply(...)</code> for a custom per-customer summary (e.g. total spend and number of active months).
3.15	Grouping by time	Parse <code>InvoiceDate</code> to datetime, set it as the index, and <code>resample</code> monthly/weekly to chart the revenue trend.

PHASE E · Combine

#	Technique	What to do with the retail data
3.20	Concatenating	Split your clean data into two periods (first vs second half of the year) and <code>concat</code> them back — simulating the merge of two monthly extracts.
3.21	Merging	Use <code>merge</code> to add a <code>region</code> column from your lookup. Compare inner vs left joins and note which countries fail to match.

Part 2 Answer the business questions

Using your **cleaned, completed-sales** dataset, answer each question with code, a number or short table, and one or two sentences of interpretation a non-technical manager could follow.

1. **Seasonality.** What was total revenue for the year, and how does it break down by month? When is the peak trading season, and by how much does it lift sales?
2. **Best sellers.** Which 10 products earned the most revenue, and which 10 sold the most units? Are they the same list and what does any difference tell you about pricing?
3. **Markets.** Outside the UK, which countries (and which regions, using your merged lookup) are most valuable by revenue and by number of distinct customers? Where would you expand?
4. **Customer concentration.** Who are the top 10 customers by total spend, and what share of total revenue do the top 1% of customers account for? Is this a wholesale-driven business?

5. **Order value.** What is the average order value (revenue per invoice), and how does it differ between UK and non-UK customers?
6. **Returns & cancellations.** How common are cancellations/returns, by count and by value? Which products or customers are most associated with them?
7. **Data-quality memo.** What share of the raw rows did you remove or repair (missing IDs, duplicates, cancellations, bad prices)? What assumptions did you make, and would you trust this dataset for a board report? Why or why not?

Deliverables

- **Cleaning notebook** a commented `.ipynb` that performs every step above, with each of the 21 techniques labelled by its number.
- **Cleaned dataset** exported to `clean_online_retail.csv`, plus a short data dictionary of your final columns.
- **Findings summary** one page answering the seven business questions, with at least two charts (e.g. monthly revenue trend, top markets).
- **Cleaning decisions log** a short list of the judgment calls you made (what you dropped/kept/changed) and why.

Grading rubric (100 points)

Criterion	Points
Loading & data profiling done correctly (encoding, shape, missingness identified)	10
All techniques demonstrated, correctly used, and clearly labelled	30
Cleaning decisions are sound and explicitly justified (raw kept separate from clean)	20
Business questions answered correctly, with evidence and clear interpretation	25
Clarity, documentation, and reproducibility of the notebook	10

Deliverables

- Your work is delivered in two places: a **GitHub repository** (all the technical work) and a **zipped submission to Canvas**.

GitHub repository must contain:

- **The cleaning notebook** (`.ipynb`) that performs every step, with each of the 21 techniques clearly labelled by its number.
- **The cleaned dataset**, exported to `clean_online_retail.csv`, plus a short data dictionary of your final columns.
- **A findings summary** one page answering the seven business questions, with at least two charts.

- **Cleaning decisions log** a short list of the judgment calls you made (what you dropped, kept, or changed) and why.
- **A README.md** explaining what the project does, how to download the dataset from Kaggle, how to run the notebook, and a brief summary of your findings.
- **Requirements.txt** listing the Python libraries (and versions) needed to reproduce your work.

Submit to Canvas:

- The **link to your GitHub repository**, and
- A **single zipped folder** of the repository.